



FEU INSTITUTE OF TECHNOLOGY

ELECTRICAL AND ELECTRONICS ENGINEERING DEPARTMENT

ADVANCED DIGITAL SIGNAL PROCESSING LAB

(ECE0055L/TT041)

FINAL PROJECT

**CELLULAR PHONE IDENTIFICATION USING
CONVOLUTIONAL NEURAL NETWORKS IN MATLAB**

Cellphone Identifier Using CNN

⌵ △ - ✕

Classification Result: blackberrycingular
Accuracy: 94.92%



Load Image

SUBMITTED BY:

Chia, John David

Macalinao, Ronald Siegfried

Tajaros, Val Rique

SUBMITTED TO:

Dr. Pocholo James M. Loresco

Course Adviser

I. INTRODUCTION

Accurately identifying mobile phones based on their distinctive features is becoming more and more crucial in the ever-changing industry of telecommunications.

Numerous uses exist for cellular phone identification, such as strengthening network management, customizing user experiences, and strengthening security protocols.

Conventional phone identification techniques sometimes depend on rudimentary statistical models or manual inspections, neither of which may be scalable or reliable for big datasets.

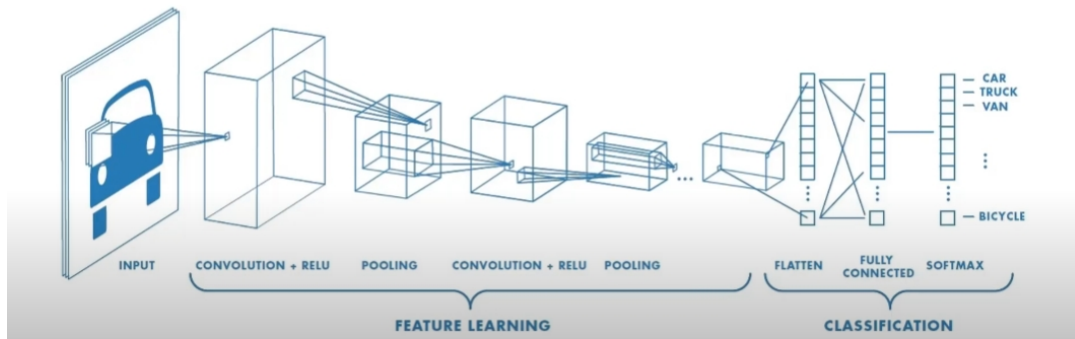
This paper describes a novel way of identifying cell phones with MATLAB-implemented Convolutional Neural Networks (CNNs). Because CNNs are a family of deep learning models that can automatically learn and extract hierarchical features from raw input data, they have shown remarkable success in image recognition tasks. This research attempts to create a dependable and effective method for detecting cell phones based on their unique patterns by utilizing the potent capabilities of CNNs. By the end of this guide, you will have a deep understanding of the underlying principles of CNNs, as well as hands-on experience in applying these techniques to solve real-world identification challenges.

II. DISCUSSION

The Cellular Phone Identifier requires loading an image that is not part of the training process to identify. After loading the image, it will begin to train the datasets in the folder name "*phonedata*". You are then prompted to enter an image file, for the sake of our demonstration the "*fortest*" folder has image files that are not used for the training data of the CNN and have a selection of phone images from the 8 classes.

The Cellular Phone Identifier identifies the type of Cellular Phone image loaded from the 8 classes of phones that we used for training. The software also uses the pre-trained CNN "*resnet50*".

Convolutional Neural Network



8 Class Types:

1. Blackberry Cingular



2. Infinix Note 12



3. iPhone 12



4. iPhone 15



5. Samsung A04



6. Samsung A20



7. Samsung A50



8. Samsung Keypad (Keystone)



Hyperparameters:

ImageInputLayer with properties:

```
        Name: 'input_1'
      InputSize: [224 224 3]
    SplitComplexInputs: 0

Hyperparameters
  DataAugmentation: 'none'
    Normalization: 'zerocenter'
  NormalizationDimension: 'auto'
    Mean: [224x224x3 single]
```

ClassificationOutputLayer with properties:

```
        Name: 'ClassificationLayer_fc1000'
      Classes: [1000x1 categorical]
    ClassWeights: 'none'
      OutputSize: 1000

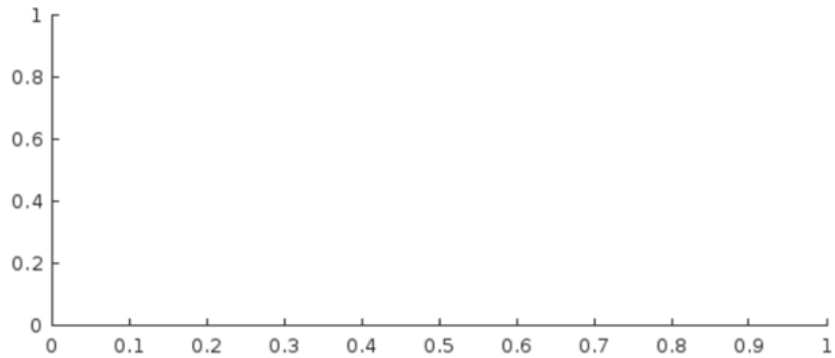
Hyperparameters
  LossFunction: 'crossentropyex'
```

The Cellular Phone Identifier was able to achieve training accuracies in the range of 85%-95% most of the time in the high 90's. The Cellular Phone Identifier software has a GUI with a button to load the image and an axes to display the input image and text prompts to display the result classification.

III. USER MANUAL

Cellphone Identifier Using CNN

🔍 △ - ✕



Load Image

1

Cellphone Identifier Using CNN

🔍 △ - ✕

Classification Result: samsungkeystone
Accuracy: 92.80%

2

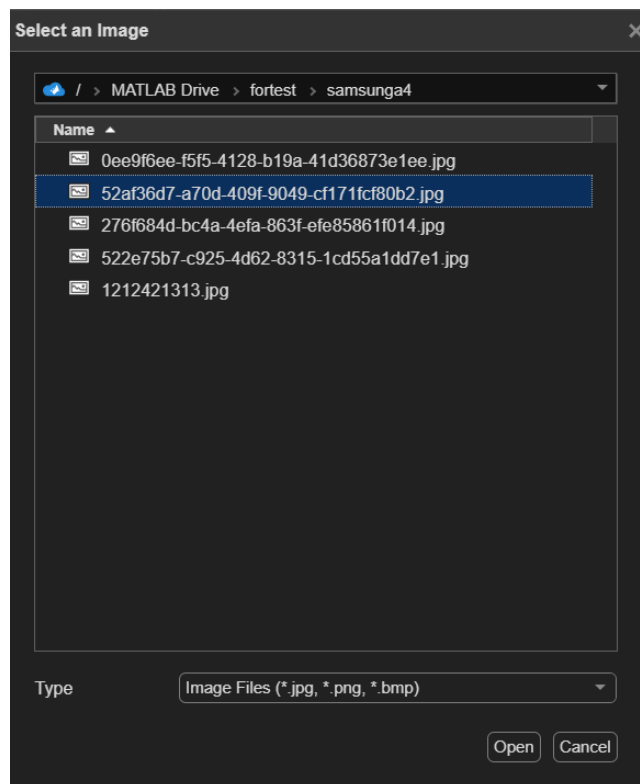
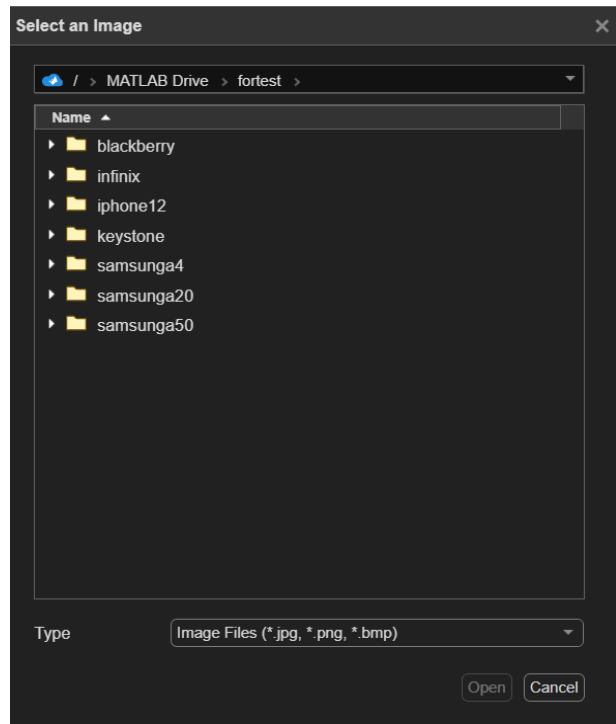
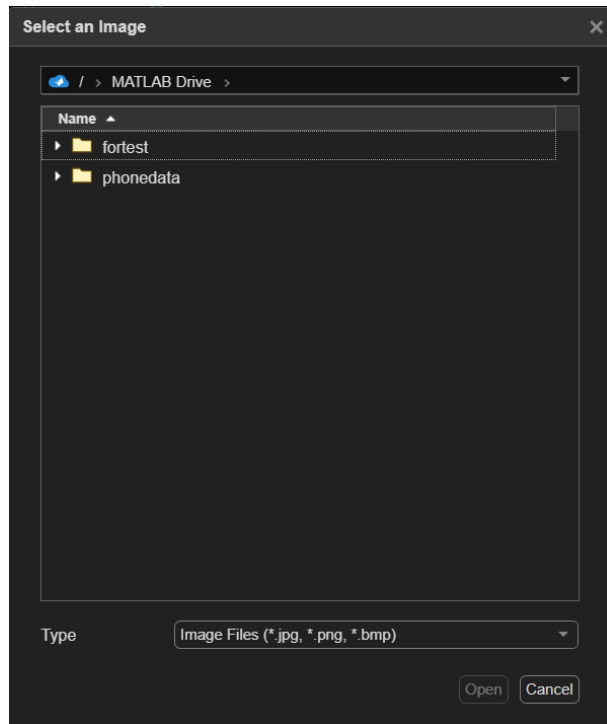


3

Load Image

1. Load Image button

Once clicked the software would prompt you to select an image file to classify a jpg, png, or bmp file.



2. Label TextBox (Classifications Results)

Once the image has been processed and identified the label textbox displays the classification result and the training accuracy of the dataset.

Classification Result: samsungkeystone
Accuracy: 92.80%



3. Axes (Image Input)

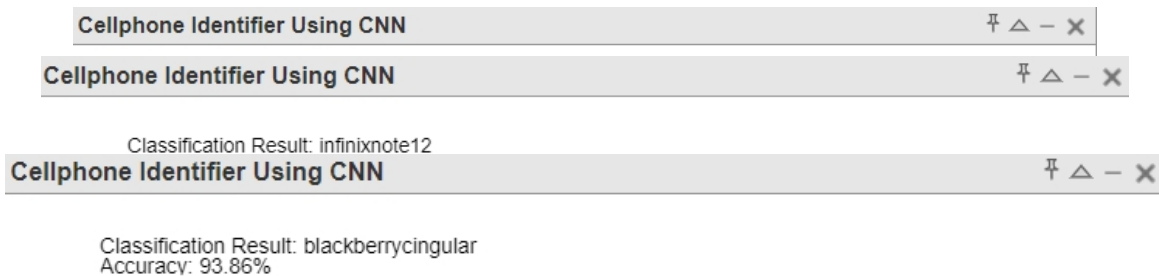
Once an image has been loaded the input image would be displayed in the axes.



To load
classify simply load an image again with the Load Image button.

another image to

SAMPLE OUTPUTS:



Load Image

Cellphone Identifier Using CNN

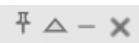


Classification Result: samsunga50
Accuracy: 92.16%



Load Image

Cellphone Identifier Using CNN

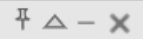


Classification Result: samsunga4
Accuracy: 95.13%



Load Image

Cellphone Identifier Using CNN



Classification Result: iphone12
Accuracy: 92.58%



Load Image

Cellphone Identifier Using CNN



Classification Result: iphone15
Accuracy: 93.64%



Load Image

Cellphone Identifier Using CNN



Classification Result: samsunga20
Accuracy: 91.53%



Load Image

CODE:

```
function CellphoneIdentifier
    % Create the figure
    hFig = figure('Position', [100, 100, 600, 400], 'MenuBar', 'none', ...
        'Name', 'Cellphone Identifier Using CNN', 'NumberTitle',
        'off', ...
        'Resize', 'off');

    % Create an axes to display the image
    hAxes = axes('Parent', hFig, 'Units', 'pixels', ...
        'Position', [50, 100, 500, 200]);

    % Create a push button to load the image
    hButton = uicontrol('Parent', hFig, 'Style', 'pushbutton', ...
        'String', 'Load Image', ...
        'Position', [250, 20, 100, 40], ...
        'Callback', @loadImageCallback);

    % Create a text label to display the classification result
    hText = uicontrol('Parent', hFig, 'Style', 'text', ...
        'Position', [50, 360, 1500, 20], ...
        'HorizontalAlignment', 'left', ...
        'FontSize', 9, 'BackgroundColor', 'white');
```

```

function loadImageCallback(~, ~)
    % Open a file selection dialog
    [filename, pathname] = uigetfile({'*.jpg;*.png;*.bmp', 'Image Files (*.jpg, *.png, *.bmp)'; ...
                                     '.*', 'All Files (*.*)'}, ...
                                     'Select an Image');

    % Check if a file was selected
    if isequal(filename, 0)
        return; % User canceled the file selection
    end

    % Read and display the image
    img = imread(fullfile(pathname, filename));
    imshow(img, 'Parent', hAxes);

    % Classify the image using your CNN
    [result, accuracy] = classifyImageWithCNN(img);

    % Display the classification result
    resultStr = sprintf('Classification Result: %s\nAccuracy: %.2f%%',
result, accuracy*100);
    set(hText, 'String', resultStr);
end
end

function [result, accuracy] = classifyImageWithCNN(img)
    outputFolder = fullfile('phonedata');
    rootFolder = fullfile(outputFolder);

    categories = {'blackberrycingular', 'infinixnote12', ...
        'iphone12', 'iphone15', 'samsunga4', 'samsunga20', 'samsunga50',
'samsungkeystone'};

    %image data store
    imds =
imageDatastore(fullfile(rootFolder, categories), 'LabelSource', 'foldernames');

    tbl = countEachLabel(imds);
    minSetCount = min(tbl(:,2));

    imds = splitEachLabel(imds, minSetCount, 'randomize');
    countEachLabel(imds);

    net = resnet50();

    net.Layers(1);

```

```

net.Layers(end);

%training
numel(net.Layers(end).ClassNames);
[trainingSet, testSet] = splitEachLabel(imds, 0.3, 'randomize');

%resizing
imageSize = net.Layers(1).InputSize;

augmentedTrainingSet = augmentedImageDatastore(imageSize, trainingSet,
'ColorPreprocessing', 'gray2rgb');

augmentedTestSet = augmentedImageDatastore(imageSize, testSet,
'ColorPreprocessing', 'gray2rgb');

%convolution
% w1 = net.Layers(2).Weights;
% w1 = mat2gray(w1);

featureLayer = 'fc1000';
trainingFeatures = activations(net, augmentedTrainingSet, featureLayer,
'MiniBatchSize', 32, 'OutputAs', 'columns');

trainingLables = trainingSet.Labels;
classifier = fitcecoc(trainingFeatures, trainingLables, 'Learner',
'Linear', 'Coding', 'onevsall', 'ObservationsIn', 'columns');

testFeatures = activations(net, augmentedTestSet, featureLayer,
'MiniBatchSize', 32, 'OutputAs', 'columns');

predictLabels = predict(classifier, testFeatures, 'ObservationsIn',
'columns'); %predicted labels

testLables = testSet.Labels; %actual labels

%confusion matrix
confMat = confusionmat(testLables, predictLabels);
confMat = bsxfun(@rdivide, confMat, sum(confMat,2));

accuracy = mean(diag(confMat)); %accuracy

ds = augmentedImageDatastore(imageSize, img, 'ColorPreprocessing',
'gray2rgb');

imageFeatures = activations(net, ds, featureLayer, 'MiniBatchSize', 32,
'OutputAs', 'columns');

```



```
    result = predict(classifier, imageFeatures, 'ObservationsIn', 'columns');  
end
```

LINK FOR THE APP & DATASET:

<https://drive.google.com/drive/folders/1T4TD2-jPptugZpnw-RyzEI14M5vfybck?usp=sharing>

IV. CONCLUSION

Creating the app to recognize obscure images is the most challenging part of the project. As modern smartphones look the same at the front, as there aren't any more buttons in front of smartphones, the system sometimes misclassifies them. A larger training dataset for all classes would help raise the accuracy, as there are only 85 images per class. Some other difficulties that we have faced are the creation of the GUI, as with MATLAB online, the tool environment is not the same as the desktop counterpart of MATLAB, and the reference codes are not matching for us to integrate the GUI properly with the classifier. Phone classification on modern phones would more likely need to be images of the back parts of the phones rather than the front, as they all look the same and only differ in size, e.g., iPhones and Samsungs. It also faced challenges with images that are from the internet and have many differences from the training dataset. All in all, the software app classifier was able to achieve accuracy as high as 95%, so we can confidently say that the software works as intended.